BDF - Customized Product Documentation

BDF Bakery Product Documentation: Racer2 V3

1. Introduction to Racer2 V3 for BDF Automation

Racer2 V3 is a powerful AI Core service designed to enhance quality control and production monitoring for BDF's bakery automation lines. It integrates advanced camera acquisition, image stitching, live streaming, and optional AI-driven inspection to ensure consistent product quality and operational efficiency.

This documentation focuses on how Racer2 V3 can be configured, operated, and monitored to support your automated bakery production.

Purpose

Racer2 V3 streamlines your production monitoring by offering:

- **Line-Scan Acquisition:** High-speed capture of bakery products as they move along the production line.
- **Frame Stitching:** Combining images from multiple cameras into a single, comprehensive view of each product.
- **Live Streaming:** Real-time visual feedback for operators to monitor the production process.
- **AI-Assisted Inspection:** Automated detection of defects or inconsistencies, crucial for maintaining BDF's high-quality standards.

Core Capabilities for BDF Production

Racer2 V3 provides key functionalities tailored for bakery automation:

- **Comprehensive Product Scanning:** Utilizes dual Basler line cameras to acquire detailed images across the width of your bakery products.
- **Seamless Image Integration:** Stitches individual camera outputs into one complete, high-resolution image for thorough inspection.
- **Real-time Visual Feedback:** Offers browser-friendly JPEG capture and live WebSocket streaming, allowing operators to see products as they are processed.
- **Flexible Camera Control:** Supports local camera access (direct control), remote camera access (for distributed setups), and emulated camera generation (for development and testing).
- **Automated Quality Inspection:** Integrates advanced AI models (RF-DETR or YOLO) for precise defect detection and product verification.
- **Consistent Color Analysis:** Provides per-object color analysis, buffering recent data to monitor and maintain product appearance.

- **External Integration:** Enables forwarding of frames for external analysis (`POST /analyze`) and local, file-driven inference (`POST /inference`).

Operating Modes for BDF Production

Racer2 V3 offers flexible operating modes to fit various BDF production environments:

- **Local Camera Mode:** The main application directly controls the Basler cameras. Ideal for integrated setups where the processing unit is close to the production line.
- **Remote Camera Mode:** The main application polls a separate, dedicated camera service over the network. This allows for placing camera hardware closer to the production line while running the main application (with AI and web interface) on a different machine.
- **Emulated Camera Mode:** The application generates synthetic product images for debugging, UI validation, and development without requiring physical cameras. This is useful for testing without impacting live production.

2. Production Monitoring and Quality Control with Racer2 V3

Racer2 V3 empowers BDF operators and engineers with tools for proactive production monitoring and automated quality assurance.

Real-time Production Monitoring

Keep a close eye on your bakery products as they move through the line:

- **Live Product View:**
- **Latest Frame:** Access the most recent product image via `GET /capture`. This provides an on-demand snapshot of the production.
- **Live Stream:** Utilize the `WebSocket /ws` endpoint for a continuous, real-time video feed of products, directly viewable in the browser UI.
- **System Health Status:**
- **Dashboard (`GET /status`):** Provides a quick overview of the system's operational state, including:
- **AI Readiness:** Confirms if the AI inspection system is active and ready to analyze products.
- **Camera Readiness:** Indicates whether the cameras are properly connected and acquiring images.
- **Stream Readiness:** Verifies that the live video feed is functioning correctly.

Automated Quality Inspection

Leverage AI to automatically detect and analyze product characteristics:

- **AI-Assisted Defect Detection:**
- The system uses configurable AI models (RF-DETR or YOLO) to identify specific defects, anomalies, or quality issues in real-time on your bakery products.
- Inference results can be viewed on the live stream or through dedicated API endpoints.
- **Color Analysis for Product Consistency:**
- Racer2 V3 captures and analyzes color information for tracked objects on the production line. This is crucial for maintaining consistent product appearance and quality.

- Operators can retrieve buffered color data via `GET /color-data` or `POST /color-analysis` for detailed reports or integration with other systems.
- Color analysis data can be exported in CSV format for further review or auditing via `GET /export_color_csv`.
- **Product Alignment Adjustments:**
- Ensure optimal image stitching for accurate inspection by adjusting camera alignment.
- **View Current Alignment:** `GET /camera/alignment` retrieves the horizontal and vertical offset values.
- **Update Alignment:** `POST /camera/alignment` allows operators to fine-tune stitching parameters (like `h_offset`, `v_offset`) to perfectly combine images of products. This resets the internal image buffer, ensuring adjustments take immediate effect.

3. Configuration and Deployment for BDF Automation

Racer2 V3 is designed for flexible deployment and configuration, allowing BDF engineers to adapt it to specific production requirements.

Flexible System Configuration

Key settings influence how Racer2 V3 operates, managed through `settings.json` and `.env` files.

- **Camera Operation Mode (`CAMERA_MODE`):**
- Set to `local` for direct camera control on the main application host.
- Set to `remote` to connect to a separate camera service.
- Set to `emulated` for development or UI testing without physical cameras.
- **AI Model Selection (`AI_MODEL_NAME`):**
- Choose `rfdetr` for advanced RF-DETR-backed detection and tracking.
- Any other value will enable YOLO detection with SORT tracking for general object detection.
- **Camera Pixel Format (`CAMERA_PIXEL_FORMAT`):**
- Ensures correct interpretation of camera images (e.g., `BGR8` or `YCbCr422_8`) for accurate color representation of bakery products.
- **Pylon-Managed Camera Configuration (`CAMERA_CONFIG_BY_PYLON`):**
- Set to `true` if camera features (like pixel format, exposure, trigger) are configured externally (e.g., via Pylon Viewer) and should not be overwritten by Racer2 V3.
- **Remote Camera Service URL (`REMOTE_CAMERA_BASE_URL`):**
- Critical for `remote` mode, specifying the network address of the dedicated camera service.

Deployment Options for BDF

Racer2 V3 supports various deployment strategies to integrate seamlessly into BDF's existing infrastructure:

- **Integrated Setup (Single-Process Local):**
- Simplest for production when the same Windows machine has direct access to Basler cameras and hosts the main application.
- **Recommended:** `CAMERA_MODE=local`.
- **Distributed Setup (Split Camera and App):**

- Ideal when camera hardware must remain close to the production line, while the main application (with AI and web interface) runs on a separate machine.
- **Camera Host:** Runs `python .\camera_main.py` on port `8001`.
- **Main App Host:** Runs `python .\main.py` with `CAMERA_MODE=remote` and `REMOTE_CAMERA_BASE_URL` pointing to the camera host (e.g., `http://<camera-host>:8001`).
- **Development & Testing (Debug or Hardware-Free):**
- Use `APP_ENV=debug` and `CAMERA_MODE=emulated` for testing the UI, APIs, or AI flows without requiring physical cameras or affecting live production.
- **Docker Deployment (Main App):**
- Allows the main application to run in a container, commonly used with a dedicated Windows host running the camera service.
- **Recommended:** Windows host runs `camera_main.py`, Docker container runs the main app with `CAMERA_MODE=remote` and `REMOTE_CAMERA_BASE_URL=http://host.docker.internal:8001`.

Environment Expectations

- **Local Camera Mode:** Requires Pylon installation and direct process access to Basler cameras.
- **Remote Camera Mode:** Assumes a running camera service and stable network connectivity to `REMOTE_CAMERA_BASE_URL`.
- **Emulated Mode:** Requires no camera hardware, making it safe for frontend and integration testing.

4. Troubleshooting and API Reference for BDF Operators

This section provides quick diagnostic steps for common issues and an overview of key API endpoints for BDF operators and integration specialists.

Common Operational Issues and Solutions

When an issue arises, follow these simple steps to diagnose and resolve it quickly:

- **Server Starts But No Product Frames Arrive:**
- Check `GET /status`. Is `camera.ready` `false`?
- If `local` mode: Verify Pylon is installed, Basler cameras are connected, and no other process is using them.
- If `remote` mode: Verify `REMOTE_CAMERA_BASE_URL` is correct and the separate camera service (on port `8001`) is running and providing JPEGs via its `/capture` endpoint.
- If `emulated` mode: Ensure `CAMERA_MODE=emulated` is set.
- **Camera Busy or Unavailable:**
- Ensure Pylon Viewer or any other camera application is closed.
- Confirm no previous instance of Racer2 V3 is still running.
- A host restart might resolve driver-related issues.
- **AI Never Becomes Ready:**
- Check `GET /status` and review the `ai.phase` and `ai.message`.
- Verify `ROBOFLOW_API_KEY` or other AI credentials in your `.env` file are valid.

- Confirm `AI_MODEL_NAME` is set to your intended model. Model initialization can take time.
- **Color Endpoints Return Empty Data:**
- Verify `DETECTION_AND_TRACK=true` in your configuration.
- Ensure the AI model is ready and actively processing tracked objects.
- Note that `GET /color-data` and `POST /color-analysis` clear the buffer after returning data.
- **General Diagnostic Sequence for Operators:**

1. Call `GET /status` to check overall system health.

2. Confirm `camera.ready` is `true`.

3. Confirm `GET /capture` returns a JPEG image.

4. Verify the browser page on `/` loads and displays frames correctly.

5. If AI is enabled, wait for `ai.phase` to become `ready`.

6. Then test AI-related features like `POST /analyze` or color analysis.

Key API Endpoints for BDF Integration

For seamless integration with BDF's existing systems and for advanced monitoring, Racer2 V3 exposes a set of REST and WebSocket APIs.

Main Service (http://localhost:8000)

- **`GET /`**: Access the browser-based operator interface.
- **`GET /status`**: Retrieve structured runtime state (AI, camera, stream readiness).
- **`GET /capture`**: Obtain the latest JPEG frame from the production line.
- **`POST /analyze`**: Send the latest frame for external inference, useful for advanced quality checks.
- **`GET /camera/alignment`**: Get current camera stitching alignment values.
- **`POST /camera/alignment`**: Update camera stitching alignment settings.
- **`GET /color-data`**: Retrieve the current color analysis buffer (and clear it).
- **`POST /color-analysis`**: Get product-shaped color analysis output (and clear the buffer).
- **`POST /inference`**: Run local AI inference on an image from disk, useful for offline analysis.
- **`WebSocket /ws`**: Stream raw or AI-annotated JPEG frames in real-time.

Dedicated Host Camera Service (http://localhost:8001 - when running in `remote` mode)

- **`GET /status`**: Get camera-specific readiness and state.
- **`GET /camera/alignment`**: Retrieve camera-side stitch alignment.
- **`POST /camera/alignment`**: Update camera-side stitch offsets.
- **`GET /capture`**: Get the latest raw JPEG frame from the camera service.
- **`WebSocket /ws`**: Stream raw JPEG frames from the camera service.

BDF - Bakery Automation